

Automating Secret Management with Vault Namespaces and External Secrets

Wilmer Moina & Diego Fernández

2025-04-27

Table of Contents

1. Introduction	1
2. Seminar Objectives	2
2.1. Key Outcomes	2
3. Environment and Tooling	3
3.1. Tools Used	3
3.2. System Requirements	3
4. Platform Foundation: K3d, Vault, ESO, and Keycloak Setup	4
4.1. Provisioning the K3s Cluster	4
4.2. Installing Vault	5
4.3. Preparing Kubernetes Namespaces and Workloads	9
4.4. Configuring Vault	10
4.5. Installing External Secrets Operator (ESO)	15
4.6. Deploying Keycloak	19
5. Deploying a Go Application with Vault, ESO, and Keycloak Integration	22
5.1. Create Realm and Client in Keycloak	22
5.2. Keycloak Configuration Validation (Optional)	24
5.3. Deploying Sentinel-1 API with Vault, ESO, and Keycloak Integration	26
5.4. Verifying the End-to-End Platform Setup	28
6. Best Practices and Security Considerations	32
7. References	33
8. List of figures	34

Chapter 1. Introduction

HashiCorp Vault is a highly flexible tool for managing secrets, encryption keys, and access policies. With the introduction of *Vault Namespaces*, organizations can isolate teams, environments, or tenants with fully independent Vault environments, all within the same cluster.

On the other hand, the *External Secrets Operator (ESO)* allows Kubernetes-native integration with external secret stores, such as Vault, AWS Secrets Manager, and others, enabling secure and declarative secret management.

This seminar explores how to:

- Provision a K3s cluster.
- Deploy Vault with multiple namespaces.
- Use External Secrets Operator to sync and inject secrets into different Kubernetes namespaces.

Chapter 2. Seminar Objectives

The main goal of this seminar is to teach students how to automate secure secret management in Kubernetes using Vault and External Secrets.

2.1. Key Outcomes

- Deploy a lightweight K3s cluster locally using `k3d`.
- Install HashiCorp Vault using Helm with support for multiple namespaces.
- Configure different Kubernetes namespaces for Dev, QA, and Prod.
- Deploy one Keycloak instance per namespace, each with its own Realm and Client.
- Use External Secrets Operator to:
 - Fetch secrets from Vault.
 - Inject them into Kubernetes workloads.
 - Automatically sync updates when Vault values change.
- Automate this setup using Terraform and Ansible.

Chapter 3. Environment and Tooling

3.1. Tools Used

- Docker (with Docker Desktop on macOS)
- k3d (K3s in Docker)
- Helm
- Terraform
- Ansible
- External Secrets Operator
- HashiCorp Vault

3.2. System Requirements

- macOS or Ubuntu with at least 8 GB RAM
- Docker installed and running
- Internet access to pull container images and charts

Chapter 4. Platform Foundation: K3d, Vault, ESO, and Keycloak Setup

This section covers the essential components for secure identity and secrets management in Kubernetes.

4.1. Provisioning the K3s Cluster

In this section, we will provision a local Kubernetes cluster using **k3d**, a lightweight wrapper that runs K3s inside Docker containers. This approach is particularly suitable for local development environments, hands-on labs, and CI pipelines due to its simplicity and speed.

4.1.1. Step 1: Install K3d



- For installation instructions, refer to the official k3d documentation: [k3d](#).
- To install **kubect1**, follow the official Kubernetes documentation: [kubect1](#).
- The Kubernetes version used in this seminar is **v1.31.5+k3s1**.

Verify the installation:

```
k3d version
```

Expected output should look similar to:

```
k3d version v5.8.3  
k3s version v1.31.5-k3s1 (default)
```

4.1.2. Step 2: Create the K3s Cluster

We'll create a cluster with one server node and two agent nodes, exposing ports 443 and 80 for the load balancer.

```
k3d cluster create workshop \  
  --servers 1 \  
  --agents 2 \  
  --port "443:443@loadbalancer" \  
  --port "80:80@loadbalancer"
```

This command:

- Creates a cluster named **workshop**.
- Starts 1 server (control-plane) and 2 agents (workers).

- Exposes port 443 from the load balancer to the host.
- Disables Traefik to avoid conflicts with custom ingress controllers.

4.1.3. Step 3: Verify Cluster Status

```
kubectl get nodes
```

Expected output:

NAME	STATUS	ROLES	AGE	VERSION
k3d-workshop-agent-0	Ready	<none>	11s	v1.31.5+k3s1
k3d-workshop-agent-1	Ready	<none>	11s	v1.31.5+k3s1
k3d-workshop-server-0	Ready	control-plane,master	18s	v1.31.5+k3s1

The K3s cluster is now ready for deploying Vault and other components.

To temporarily stop the cluster and free up system resources, you can stop the underlying Docker containers:

```
k3d cluster stop workshop
```



To bring the cluster back online later:

```
k3d cluster start workshop
```

This is especially useful during multi-day workshops or when switching between tasks, as it avoids recreating the cluster from scratch.

4.2. Installing Vault

In this step, we will install HashiCorp Vault in the Kubernetes cluster using Helm. Vault will serve as our central secret management system. We will use the official Helm chart and run it in "dev" mode initially, just to validate the deployment and make quick iterations.



Recommended Resources:

- [What is Vault?](#)
- [Vault Helm chart](#)
- [Helm Installation](#)

4.2.1. Add the HashiCorp Helm Repository

```
helm repo add hashicorp https://helm.releases.hashicorp.com
```

```
helm repo update
```

4.2.2. Deploying Vault in HA Mode

In this section, we will deploy Vault in High Availability (HA) mode using the integrated Raft storage backend. This setup provides a more realistic and production-like experience, suitable for multi-node clusters and secret replication scenarios.

We start by installing Vault using the official Helm chart, with a custom `values.yaml` file that enables HA mode and configures Raft storage with `retry_join` directives.

```
helm upgrade --install vault hashicorp/vault \
  --namespace vault \
  --create-namespace \
  -f helm/vault/config.yaml
```

The configuration:

- Deploys 3 Vault replicas.
- Enables Raft storage with automatic `retry_join`.
- Disables TLS for simplicity (not recommended for production).
- Exposes the Vault UI using a LoadBalancer service.

4.2.3. Wait for the Pods to Be Ready

Check that all Vault pods are running:

```
kubectl get pods -n vault
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE
vault-0	1/1	Running	0	2m
vault-1	1/1	Running	0	2m
vault-2	1/1	Running	0	2m

4.2.4. Initialize the Vault Cluster

Vault must be initialized before it can be used. This step generates the master key, which is split into multiple unseal keys.

We'll use a simplified setup with only **1 unseal key** required:

```
kubectl exec -it vault-0 -n vault -- vault operator init -key-shares=1 -key-
```

```
threshold=1
```



Vault uses Shamir's Secret Sharing algorithm to split a master key into multiple parts (shares). - **key-shares** is the total number of unseal keys generated. - **key-threshold** is the number of keys required to unseal the Vault.

For this seminar, we simplify by using 1 key out of 1 (**1-of-1**), which is **not secure for production**, but ideal for learning purposes.

Make sure to **copy and store the unseal key and root token** that are printed. You'll need them in the next step.

4.2.5. Unseal All Vault Nodes

Each Vault pod must be manually unsealed (unless auto-unseal is configured). You will use the same unseal key generated during the initialization step.

To simplify this process, we recommend storing the unseal key in a shell variable:

```
export VAULT_UNSEAL_KEY="your-unseal-key-here"
```

Replace **"your-unseal-key-here"** with the key printed during **vault operator init**.

Then unseal each pod using:

```
kubectl exec -it vault-0 -n vault -- vault operator unseal $VAULT_UNSEAL_KEY
kubectl exec -it vault-1 -n vault -- vault operator unseal $VAULT_UNSEAL_KEY
kubectl exec -it vault-2 -n vault -- vault operator unseal $VAULT_UNSEAL_KEY
```

After all nodes are unsealed, you can verify the cluster status by running:

```
kubectl exec -it vault-0 -n vault -- vault status
```

This will show whether Vault is sealed, the current HA mode, the active node, and peer connectivity.

4.2.6. Accessing the Vault UI via Ingress

To access the Vault Web UI using an Ingress controller (e.g. Traefik), we expose the Vault service externally by deploying a custom Ingress using our own Helm chart.

1. Make sure an Ingress controller like Traefik is deployed in your cluster.
2. Deploy the Ingress pointing to the **vault-ui** service using the following Helm command:

```
helm upgrade --install vault-ui-ingress ./helm/traefik/ingress \
```

```
--namespace vault \
--set name=vault-ui-ingress \
--set namespace=vault \
--set annotations.ingressClass=traefik \
--set spec.ingressClassName=traefik \
--set spec.rules.host=vault.test \
--set spec.rules.paths[0].path="/" \
--set spec.rules.paths[0].pathType=Prefix \
--set spec.rules.paths[0].backend.service.name=vault-ui \
--set spec.rules.paths[0].backend.service.port=8200 \
--set tls.enabled=false
```

This configuration:

- Deploys a Kubernetes **Ingress** resource under the **vault** namespace
- Points to the **vault-ui** service on port **8200**
- Uses Traefik as the Ingress controller
- Does **not** enable TLS (exposes via HTTP only)
 1. Add a DNS entry in your local **/etc/hosts** file to route **vault.test** to your local Ingress controller (typically **127.0.0.1**):

```
echo "127.0.0.1 vault.test" | sudo tee -a /etc/hosts
```

1. Open your browser and go to: <http://vault.test>
2. Log in using the **Initial Root Token**.

Vault is now accessible through an Ingress route, ready to be used with the rest of your platform.

You can also access the Vault UI via port-forwarding:

```
kubectl port-forward -n vault svc/vault-ui 8200:8200
```

Then visit: <http://localhost:8200> Login using the **Initial Root Token**.

After logging in, you should see the Vault UI dashboard

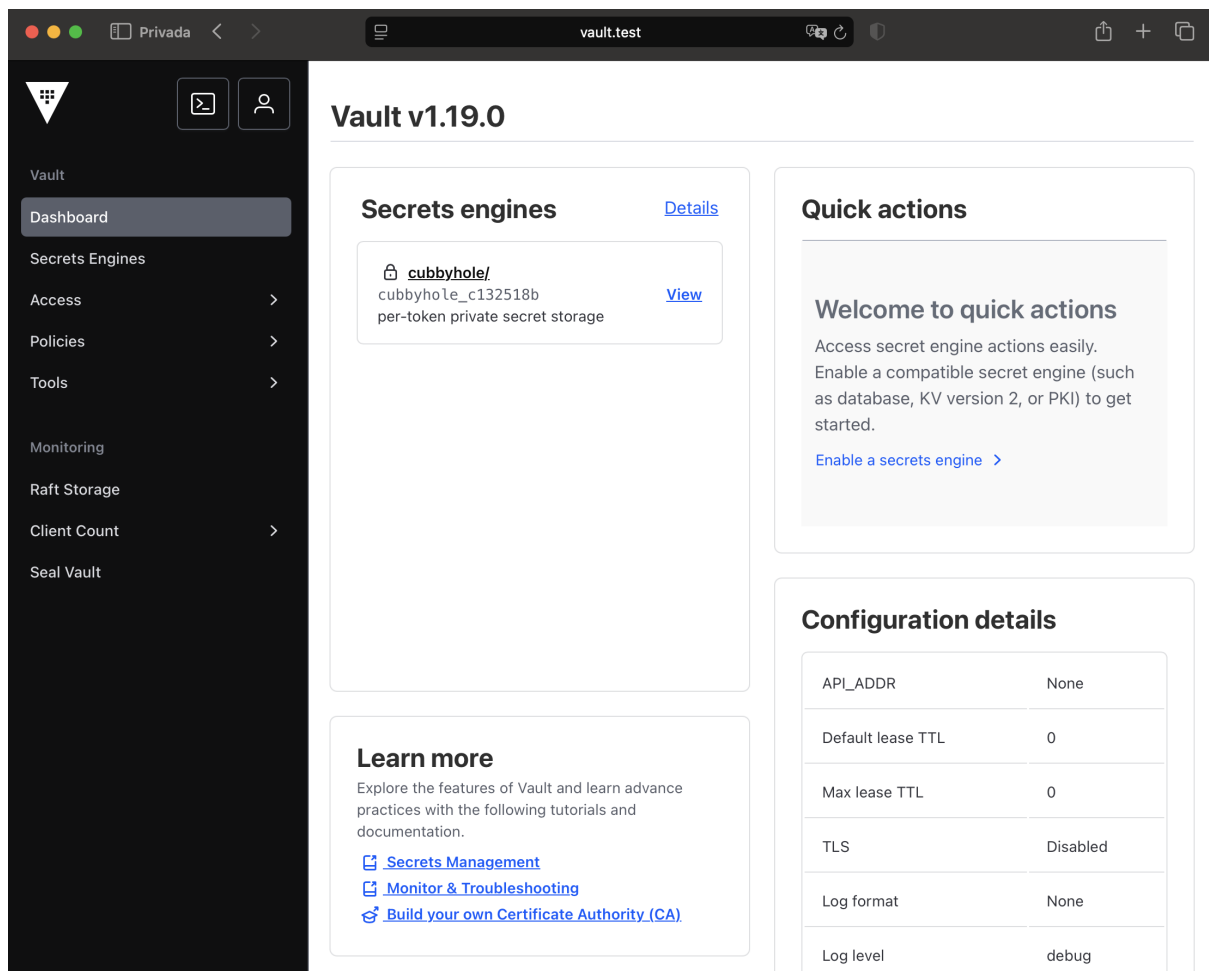


Figure 1. Vault dashboard

4.3. Preparing Kubernetes Namespaces and Workloads

Before configuring Vault, we need to prepare our Kubernetes cluster with the necessary namespaces.

Each environment (**dev**, **qa**, **prod**) will have:

- A dedicated Kubernetes namespace
- A custom ServiceAccount named **vault-sa**

This setup ensures that:

- Vault roles and policies can bind securely to service accounts per namespace
- Secrets can later be synced with External Secrets Operator (ESO) in a clean, isolated manner

4.3.1. Create Namespaces and Service Accounts

```
for ns in dev qa prod; do
  kubectl create namespace $ns
  kubectl create serviceaccount vault-sa -n $ns
done
```



We intentionally avoid using the default service account to improve security and clarity in Vault bindings.

Once these namespaces and service accounts are in place, we can proceed to configure Vault using Terraform.

4.4. Configuring Vault

With Vault running in High Availability (HA) mode, we now configure it to manage secrets for three isolated environments: `dev`, `qa`, and `prod`.

Since Vault OSS does not support namespaces (an Enterprise-only feature), we simulate environment-level isolation using a combination of:

- Separate KV secrets engines: `secret-dev`, `secret-qa`, and `secret-prod`
- Access control policies (ACLs) scoped to each environment
- Kubernetes auth roles bound to specific namespaces

All configuration is automated using Terraform for consistency, reproducibility, and version control.

4.4.1. Directory Structure

All Terraform files related to Vault configuration are located under `terraform/vault/`:

```
terraform/vault/  
├── kubernetes_auth.tf    # Enables Kubernetes auth and defines Vault roles  
├── kv_mounts.tf          # Mounts KV v2 engines for dev, qa, and prod  
├── policies.tf           # Defines ACL policies for each environment  
├── provider.tf           # Vault provider configuration using input variables  
└── variables.tf          # Input variables for Vault address and token
```

4.4.2. Mounting KV Engines

The `kv_mounts.tf` file mounts a KV v2 secrets engine for each environment using `vault_mount` resources.

These mounts appear as:

- `secret-dev/` → used exclusively by workloads in the `dev` namespace
- `secret-qa/` → used by `qa` workloads
- `secret-prod/` → dedicated to `prod`

All engines are configured with version 2 for secret versioning and soft delete support.

4.4.3. Defining Access Policies

The file `policies.tf` defines a Vault policy for each environment. Each policy allows read-only access to the corresponding KV mount, ensuring strict separation of access.

Example: `dev` policy

```
path "secret-dev/data/*" {
  capabilities = ["read", "list"]
}
```

This approach prevents workloads in one namespace from accessing secrets belonging to another.

4.4.4. Enabling Kubernetes Authentication

The `kubernetes_auth.tf` file automates the full setup of the Kubernetes auth method in Vault:

- Enables the `kubernetes` auth backend
- Configures Vault to communicate with the in-cluster Kubernetes API
- Creates one Vault `role` per environment:
- `dev` role → bound to service accounts in the `dev` namespace
- `qa` role → bound to `qa` namespace
- `prod` role → bound to `prod` namespace

This ensures that only authorized workloads can request secrets from the correct environment.

4.4.5. Providing Vault and Kubernetes Credentials

To securely configure Vault with Kubernetes authentication, we use a `.tfvars.json` file to provide all sensitive variables required by Terraform.

Create the file `secrets.auto.tfvars.json` inside `terraform/vault/` with the following content:

```
{
  "vault_addr": "http://vault.test",
  "vault_token": "hvs.YourRootTokenHere",
  "kubernetes_host": "https://kubernetes.default.svc",
  "kubernetes_ca_cert": "-----BEGIN CERTIFICATE-----\nMIID...\n-----END\nCERTIFICATE-----",
  "token_reviewer_jwt": "eyJhbGciOiJSUzI1NiIsIn..."
}
```



Terraform automatically loads all files ending in `.auto.tfvars.json`, so there's no need to specify this file manually during `terraform apply`.

4.4.6. How to Obtain the Required Values

These values can be extracted from any running pod in your cluster, such as `vault-0`.

To get the Kubernetes CA certificate:

```
kubectl exec -n vault vault-0 -- cat
/var/run/secrets/kubernetes.io/serviceaccount/ca.crt | awk '{printf "%s\\n", $0}'
```

Copy the entire output, including the `-----BEGIN CERTIFICATE-----` and `-----END CERTIFICATE-----` lines into the JSON value of `kubernetes_ca_cert`.

To get the token reviewer JWT:

```
kubectl exec -n vault vault-0 -- cat
/var/run/secrets/kubernetes.io/serviceaccount/token
```

Copy the full JWT token output and paste it into the `token_reviewer_jwt` field.



Do not commit the `secrets.auto.tfvars.json` file to version control. Add it to `.gitignore` to protect your credentials.

4.4.7. Applying the Configuration

Once the variables file is ready, apply the configuration:

```
cd terraform/vault
terraform init
terraform apply
```

Terraform will:

- Mount the three KV secrets engines
- Upload environment-specific access policies
- Enable and configure the Kubernetes authentication backend
- Create Vault roles bound to service accounts in the `dev`, `qa`, and `prod` namespaces

Your Vault instance is now fully configured for secure, isolated access from each Kubernetes environment.

4.4.8. Verifying Kubernetes Authentication to Vault

Now that Vault is configured, we can verify that a Kubernetes workload in the `dev` namespace can authenticate to Vault and access its secrets securely using the `vault-sa` service account.

Step 1: Create a test secret in Vault

First, store a secret in the `secret-dev` path using a Vault token that has the appropriate policy (e.g., the root token):

```
export VAULT_ADDR=http://vault.test
export VAULT_TOKEN=hvs.YourRootTokenHere

vault kv put secret-dev/example username="admin" password="s3cr3t"
vault kv put secret-qa/example username="admin" password="s3cr3t"
```



If you see the error:

`permission denied`

Make sure you're using a Vault token that includes the correct policy (`dev`, `qa`, or `prod`) for the secrets engine you're targeting.

Step 2: Deploy a test pod using `vault-sa`

Run the following command to deploy a temporary test pod in the `dev` namespace:

```
kubectl apply -f - <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: test-vault
  namespace: dev
spec:
  serviceAccountName: vault-sa
  containers:
  - name: tester
    image: alpine
    command: ["sh", "-c", "sleep 3600"]
    restartPolicy: Never
EOF
```

Step 3: Access the pod and install `curl` + `jq`

```
kubectl exec -it test-vault -n dev -- sh
```

Once inside the pod, install the required tools:

```
apk add --no-cache curl jq
```

Step 4: Authenticate with Vault using the pod's service account

Export the Vault address and read the pod's JWT token:

```
export VAULT_ADDR=http://vault.vault.svc:8200
export SA_TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
```

Use the token to authenticate to Vault:

```
export VAULT_TOKEN=$(curl --silent --request POST \
  --data '{"jwt": "'$SA_TOKEN'", "role": "dev"}' \
  $VAULT_ADDR/v1/auth/kubernetes/login | jq -r '.auth.client_token')

# Verify the token
echo $VAULT_TOKEN
```

The response that you receive should look similar to this:

```
{
  "request_id": "f879089e-7034-59db-c24e-ac1c9a76ea83",
  "lease_id": "",
  "renewable": false,
  "lease_duration": 0,
  "data": null,
  "wrap_info": null,
  "warnings": null,
  "auth": {
    "client_token": "hvs.CAESID_Ax0u6Jik0RIv3h2aeJLiue74a0xnqMLD0DKjqIhsNGh4KHGh2cy5kV3JJM",
    "accessor": "rU6SFISwyTziSlz83CfEyhJP",
    "policies": [
      "default",
      "dev"
    ],
    "token_policies": [
      "default",
      "dev"
    ],
    "metadata": {
      "role": "dev",
      "service_account_name": "vault-sa",
      "service_account_namespace": "dev",
      "service_account_secret_name": "",
      "service_account_uid": "492200b0-cac4-4080-a66d-f5524491e3f7"
    },
    "lease_duration": 3600,
    "renewable": true,
    "entity_id": "03223c1c-e7db-9cd4-755a-9d145baee3fb",
    "token_type": "service",
    "orphan": true,
    "mfa_requirement": null,
    "num_uses": 0
  },
  "mount_type": ""
}
```

Figure 2. Vault token issued via Kubernetes authentication using vault-sa in dev namespace

Step 5: Use the Vault token to retrieve a secret

Now that we have a Vault token, use it to access the secret:

```
curl --silent \
  --header "X-Vault-Token: $VAULT_TOKEN" \
  $VAULT_ADDR/v1/secret-dev/data/example | jq
```

If configured correctly, you should see the secret:

```
{
  "request_id": "f3e924ca-8318-7c74-7f6e-2ef63f405b5f",
  "lease_id": "",
  "renewable": false,
  "lease_duration": 0,
  "data": {
    "data": {
      "password": "s3cr3t",
      "username": "admin"
    },
    "metadata": {
      "created_time": "2025-04-11T05:22:55.373300458Z",
      "custom_metadata": null,
      "deletion_time": "",
      "destroyed": false,
      "version": 1
    }
  },
  "wrap_info": null,
  "warnings": null,
  "auth": null,
  "mount_type": "kv"
}
```

Figure 3. Secret retrieved from Vault using the token

Final Step: Clean up

Exit the pod and delete it:

```
exit
kubectl delete pod test-vault -n dev
```



You can capture this output as a screenshot to include in the lab report or Medium post as proof of successful Vault-Kubernetes integration.

4.5. Installing External Secrets Operator (ESO)

In this section, we install the External Secrets Operator (ESO), which synchronizes secrets from Vault into native Kubernetes **Secret** objects. This allows Kubernetes workloads to consume secrets managed in Vault without requiring application-level changes.

4.5.1. Recommended Resources



- [What is External Secrets Operator?](#)
- [ESO Getting Started Guide](#)

4.5.2. 1. Add the ESO Helm repository

```
helm repo add external-secrets https://charts.external-secrets.io
helm repo update
```

4.5.3. 2. Install ESO in the external-secrets namespace

```
helm upgrade --install external-secrets external-secrets/external-secrets \
--namespace external-secrets \
--create-namespace
```

This command:

- Installs ESO components (Core Controller, Webhook, Cert Controller)
- Creates the required CRDs ([SecretStore](#), [ClusterSecretStore](#), [ExternalSecret](#))
- Prepares the cluster to sync secrets from Vault

4.5.4. 3. Verify ESO pods are running

```
kubectl get pods -n external-secrets
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE
external-secrets-86f7b8cc94-jlzj9	1/1	Running	0	8m
external-secrets-cert-controller-59f9f9cd77-z4cf6	1/1	Running	0	8m
external-secrets-webhook-5fc8544c4c-dld6m	1/1	Running	0	8m



All components should report **READY 1/1** and **STATUS Running**. If not, inspect logs via `kubectl logs`.

4.5.5. 4. Create a SecretStore in each environment namespace

Each namespace ([dev](#), [qa](#), [prod](#)) must define a [SecretStore](#) resource that configures Vault access using Kubernetes authentication.

To preview the manifest using Helm templating:

```
helm template vault --namespace dev helm/external-secrets/secretStore/ --dry
-run=server | yq
```

Example output:

```
# Source: secretStore/templates/secretStore.yaml
apiVersion: external-secrets.io/v1beta1
kind: SecretStore
metadata:
  name: vault-store
  namespace: dev
spec:
  provider:
    vault:
      server: http://vault.vault.svc:8200
      path: secret-dev
      version: "v2"
      auth:
        kubernetes:
          mountPath: kubernetes
          role: dev
          serviceAccountRef:
            name: vault-sa
```

To deploy it:

```
helm upgrade --install vault --namespace dev helm/external-secrets/secretStore/
```

Verify that the SecretStore is valid:

```
kubectl get secretstore -n dev
```

Expected output:

NAME	AGE	STATUS	CAPABILITIES	READY
vault-store	48s	Valid	ReadWrite	True

4.5.6. 5. Define an ExternalSecret to sync data from Vault

An **ExternalSecret** defines how ESO should fetch data from Vault and write it into a Kubernetes **Secret**.

To preview:

```
helm template dev-credential --namespace dev helm/external-secrets/externalSecret/
--dry-run=server | yq
```

Expected output:

```
# Source: externalSecret/templates/externalSecret.yaml
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: dev-credential
  namespace: dev
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: vault-store
    kind: SecretStore
  target:
    name: synced-dev-credential
    creationPolicy: Owner
  data:
    - secretKey: username
      remoteRef:
        key: example
        property: username
    - secretKey: password
      remoteRef:
        key: example
        property: password
```

Deploy it:

```
helm upgrade --install dev-credential --namespace dev helm/external-
secrets/externalSecret/
```

Verify that it was created:

```
kubectl get externalsecret -n dev
```

Expected output:

NAME	STORETYPE	STORE	REFRESH INTERVAL	STATUS	READY
dev-credential	SecretStore	vault-store	1h	SecretSynced	True

4.5.7. 6. Verify that the secret was synced

Once synced, ESO creates a Kubernetes **Secret** with the desired values.

```
kubectl get secret synced-dev-credential -n dev -o jsonpath='{.data}' | jq
```

You should see both **username** and **password** encoded in base64.

```
{
  "password": "czNjcjN0",
  "username": "YWRtaW4="
}
```

Figure 4. Vault secret synced to Kubernetes

4.6. Deploying Keycloak

In this section, we deploy a single instance of Keycloak into the `iam` namespace using the Bitnami Helm chart. This Keycloak instance will later host one Realm per environment (`dev`, `qa`, `prod`), acting as the centralized Identity Provider for the platform.

4.6.1. Recommended Resources



- [What is Keycloak?](#)
- [Bitnami Keycloak Chart](#)
- [ArtifactHub Keycloak](#)

4.6.2. 1. Add the Bitnami Helm repository

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
```

4.6.3. 2. Deploy Keycloak in the `iam` namespace (with persistence)

We deploy Keycloak with an embedded PostgreSQL database and persistent storage enabled. This prevents data loss across pod restarts and ensures the state of realms, users, and configurations is retained.

```
helm upgrade --install keycloak bitnami/keycloak \
  --namespace iam \
  --create-namespace \
  --set auth.adminUser=admin \
  --set auth.adminPassword=admin123 \
  --set proxy=edge \
  --set service.type=ClusterIP \
  --set postgresql.primary.persistence.enabled=true \
  --set postgresql.primary.persistence.size=1Gi
```

This configuration:

- Installs Keycloak in the `iam` namespace
- Sets default admin credentials (for demo use only)
- Uses internal PostgreSQL with persistent volume

- Exposes Keycloak via **ClusterIP** (internal access only)
- Enables **proxy=edge** for UI access via port-forward



This setup is sufficient for the seminar. In production, consider using an external managed PostgreSQL service.

4.6.4. 3. Wait for the Keycloak pods to be ready

```
kubectl get pods -n iam
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE
keycloak-0	1/1	Running	0	1m

4.6.5. 4. Access the Keycloak Admin UI via Ingress

To access the Keycloak web UI, we expose it externally using an Ingress resource instead of port-forwarding. This is consistent with the approach used for Vault and better simulates a real-world deployment.

1. Deploy the Ingress pointing to the **keycloak** service on port **8080** using the following Helm command:

```
helm upgrade --install keycloak-ui-ingress ./helm/traefik/ingress \
  --namespace iam \
  --set name=keycloak-ui-ingress \
  --set namespace=iam \
  --set annotations.ingressClass=traefik \
  --set spec.ingressClassName=traefik \
  --set spec.rules.host=keycloak.test \
  --set spec.rules.paths[0].path="/" \
  --set spec.rules.paths[0].pathType=Prefix \
  --set spec.rules.paths[0].backend.service.name=keycloak \
  --set spec.rules.paths[0].backend.service.port=80 \
  --set tls.enabled=false
```

1. Add an entry to your local **/etc/hosts** so the **keycloak.test** domain resolves to your Ingress controller:

```
echo "127.0.0.1 keycloak.test" | sudo tee -a /etc/hosts
```

1. Open your browser and navigate to: <http://keycloak.test>

Login using:

- Username: **admin**
- Password: **admin123**

You now have Keycloak accessible through Ingress, consistent with how Vault is exposed.

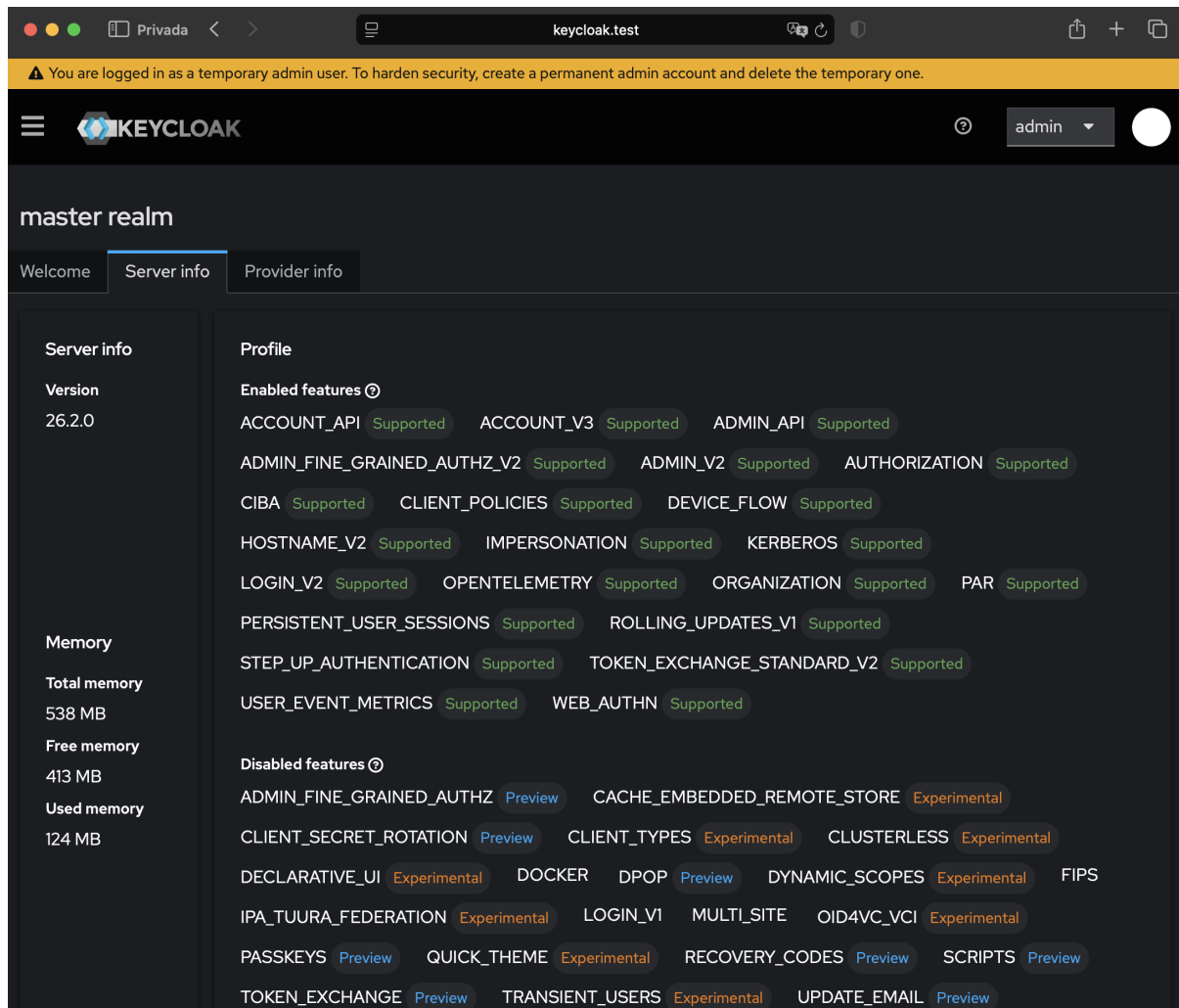


Figure 5. User interface of Keycloak

Chapter 5. Deploying a Go Application with Vault, ESO, and Keycloak Integration

In this section, we will deploy the **SENTINEL-1** Go-based API, integrating it with **HashiCorp Vault**, the **External Secrets Operator (ESO)**, and **Keycloak** as the identity provider. The API source code is publicly available at: <https://github.com/myworkshops/sentinel-1.git>.

The application will authenticate against a Keycloak client configured within dedicated realms for each environment (**dev**, **qa**, and **prod**). Each client's credentials—specifically, the **client_id** and **client_secret**—will be securely stored in the corresponding Vault KV secrets engine (**secret-dev**, **secret-qa**, **secret-prod**).

To bridge Vault and Kubernetes, a **SecretStore** and an **ExternalSecret** will be provisioned in each namespace (**dev**, **qa**, and **prod**). These resources enable ESO to dynamically retrieve credentials from Vault and project them into Kubernetes as native **Secret** objects.

Once the credentials are synchronized, the **SENTINEL-1** API will be deployed into each namespace. The application is designed to load its configuration dynamically from the mounted secrets and supports hot-reloading, ensuring real-time propagation of secret rotations without requiring restarts.

This deployment showcases a complete integration of secret management, identity federation, and secure workload configuration, following **zero-trust principles** and adhering to **cloud-native best practices**.

5.1. Create Realm and Client in Keycloak

In this step, we automate the creation of Keycloak realms, clients, and users using Ansible. This configuration is executed separately for each environment (**dev**, **qa**, **prod**) to enforce proper segregation and security boundaries.

We use the following Ansible playbooks:

- **realm.yml**: creates the Keycloak realm (e.g., **dev**, **qa**, **prod**)
- **client.yml**: defines a Keycloak client inside that realm
- **user.yml**: creates a set of users to authenticate into the application

These playbooks use a shared **values.yml** file to define reusable parameters such as client names, URIs, and user lists.



Before running the playbooks, ensure that Ansible is installed on your machine. You can install Ansible using **pip** or your system's package manager.

```
# For Ubuntu/Debian
sudo apt update
sudo apt install python3-pip -y
pip3 install --user ansible
```

```
# For macOS with Homebrew
brew install ansible
```

Verify your installation with:

```
ansible --version
```

For more, visit: https://docs.ansible.com/ansible/latest/installation_guide/intro_installation.html

5.1.1. Execute per Environment

To automate the creation of realms, clients, and users in Keycloak:

Set the environment variable **THEENV** to the desired environment (**dev**, **qa**, or **prod**):

```
export THEENV=prod
```

Run the playbooks using the **THEENV** variable to target the corresponding environment:

```
ansible-playbook realm.yml \
-e @values.yml \
-e env_work=$THEENV \
-e auth_username=admin \
-e auth_password=admin123

ansible-playbook client.yml \
-e @values.yml \
-e env_work=$THEENV \
-e auth_username=admin \
-e auth_password=admin123

ansible-playbook user.yml \
-e @values.yml \
-e env_work=$THEENV \
-e auth_username=admin \
-e auth_password=admin123
```

Repeat for each environment by setting **THEENV** accordingly.



Ensure the Keycloak instance is reachable at <http://keycloak.test> before executing these playbooks. The realms, clients, and users will be created under their respective environments.



After running the playbooks, locate the generated **client_id** and **client_secret** in

the Ansible logs. Store them manually in Vault under the corresponding path for each environment:

- `secret-dev/iam/idp_clients/sentinel-1`
- `secret-qa/iam/idp_clients/sentinel-1`
- `secret-prod/iam/idp_clients/sentinel-1`

Each secret must contain the keys: - `client_id` - `client_secret`

This ensures that credentials are securely stored for later retrieval via External Secrets Operator.

```
TASK [Debug Credentials] *****
ok: [localhost] => {
  "msg": [
    "🔑 Keycloak Client Credentials",
    "Environment      : prod",
    "Client ID         : sentinel-1",
    "Client Secret     : ETPtEL0IwWYldJ1qd5QukeSyRXY7AAge"
  ]
}
```

Figure 6. Ansible output showing client credentials

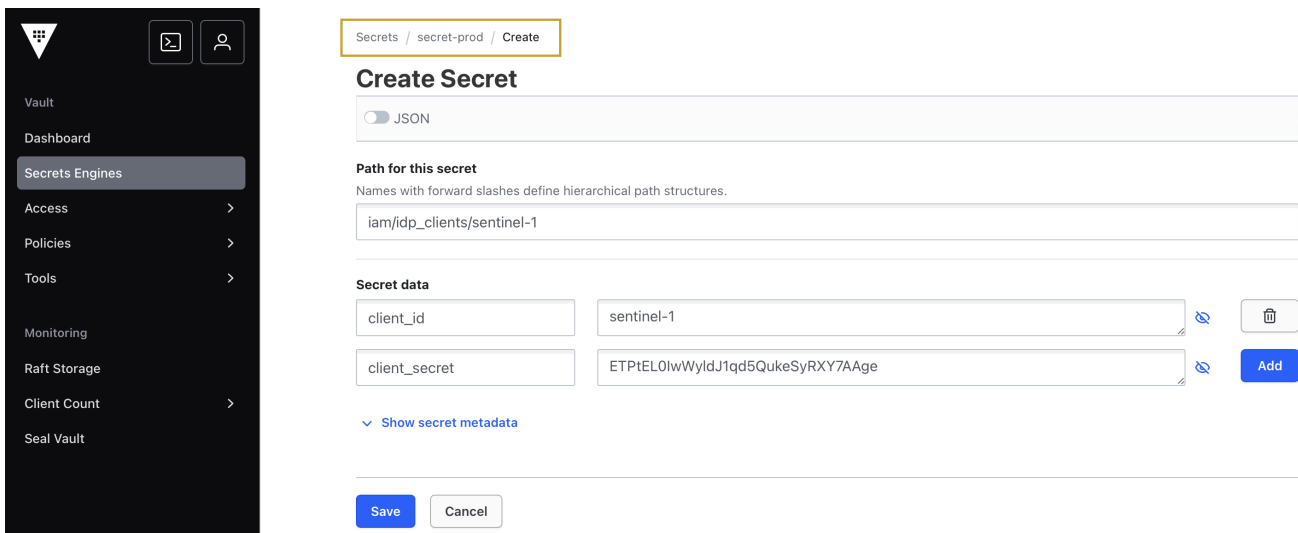


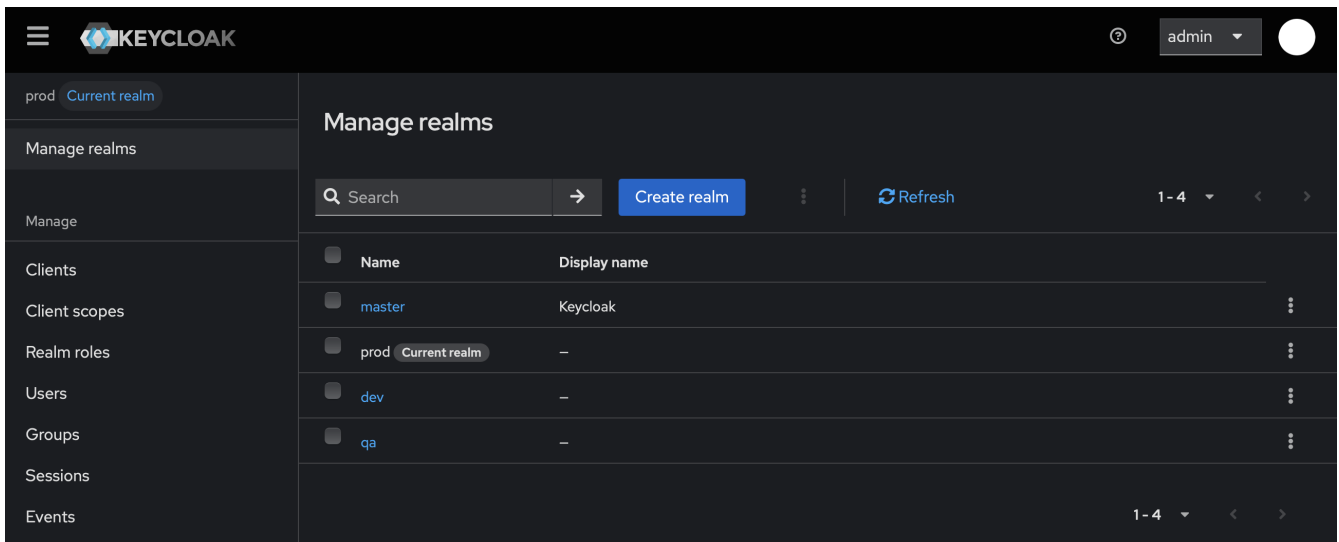
Figure 7. Credentials stored in Vault

5.2. Keycloak Configuration Validation (Optional)

After running the playbooks, you can visually confirm that the realms, clients, and users were created correctly in the Keycloak Admin UI.

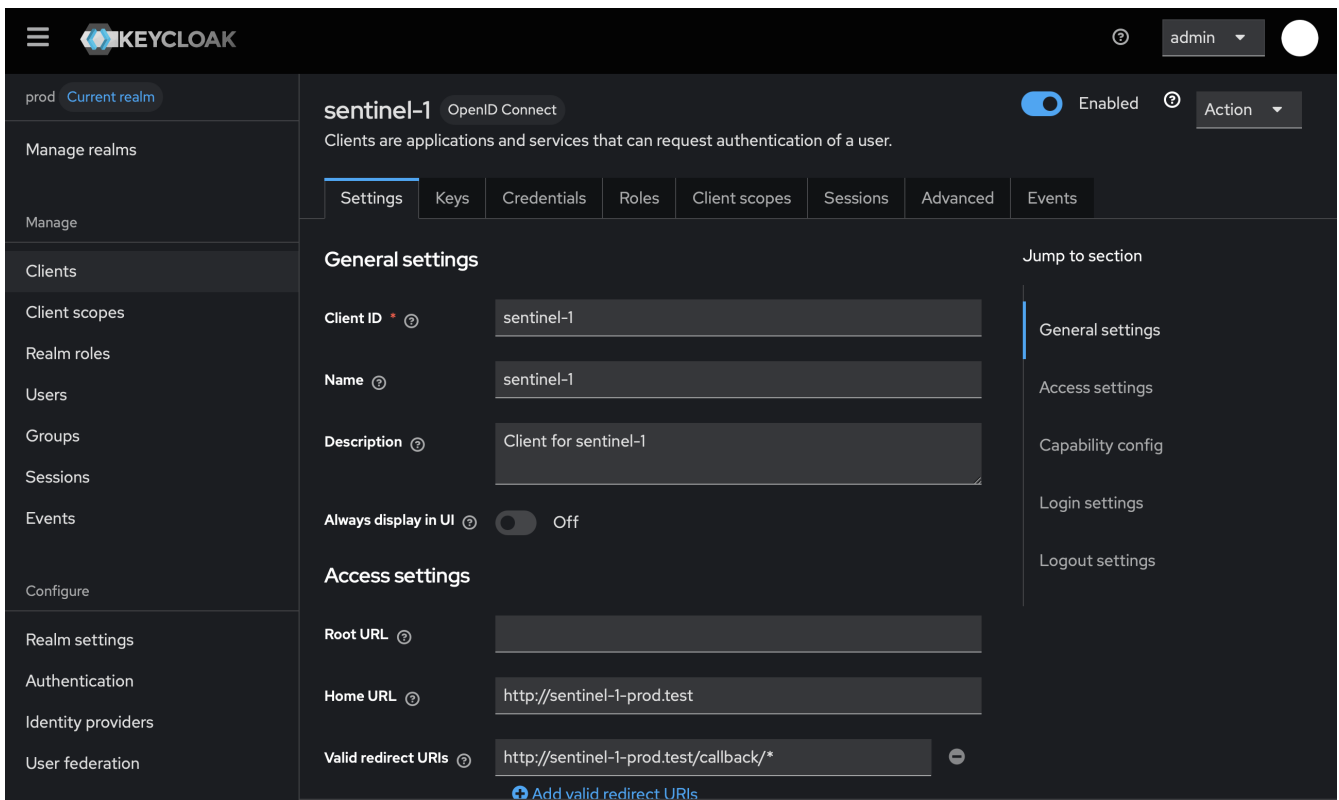
5.2.1. View Realms List

Log in to the Keycloak Admin Console (<http://keycloak.test/admin>) using the admin credentials. You should see the list of realms including: `dev`, `qa`, and `prod`.



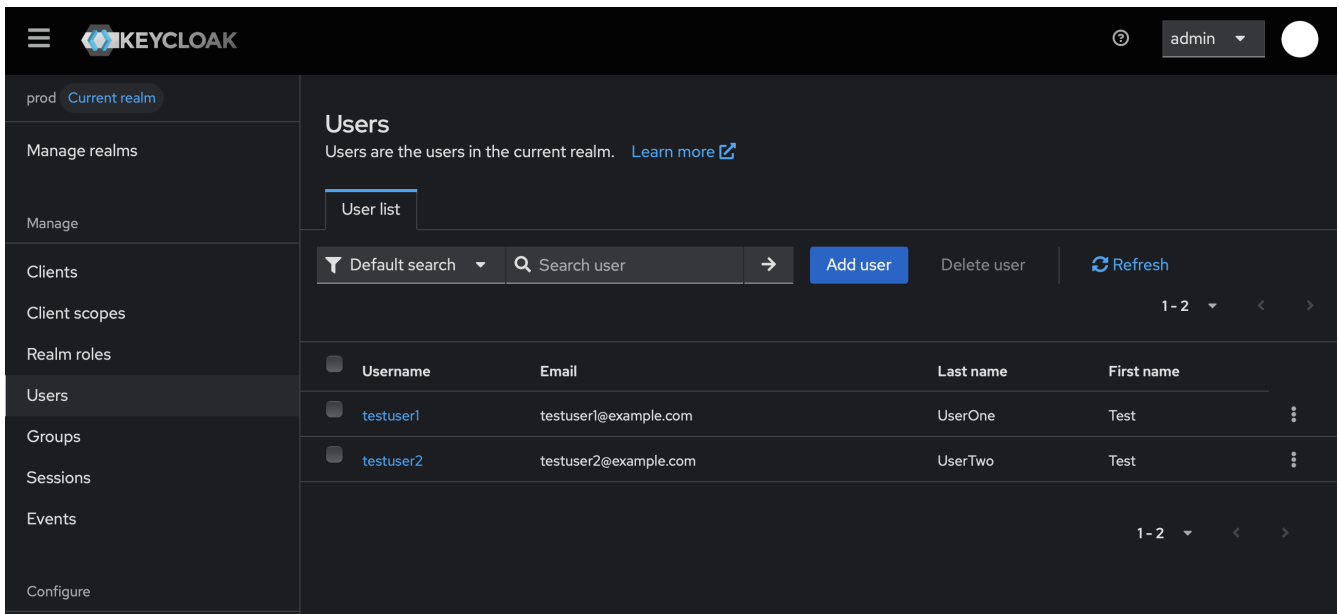
5.2.2. View Client Configuration

Within any realm (e.g. **prod**), go to **Clients** → **sentinel-1**. This shows the client details including client ID, secret, redirect URIs, and access settings.



5.2.3. View Created Users

Navigate to **Users** within the selected realm to confirm that the test users were successfully provisioned.



These visual checks serve as a quick validation that the automation executed correctly and that your identity layer is correctly configured per environment.

5.3. Deploying Sentinel-1 API with Vault, ESO, and Keycloak Integration

In this section, we will deploy the Sentinel-1 API into the `dev`, `qa`, and `prod` namespaces. Secrets (`client_id` and `client_secret`) will be synchronized from Vault into Kubernetes using the External Secrets Operator (ESO).

We will automate everything using a `for` loop to avoid repetition.

5.3.1. Step 1: Deploy SecretStore and ExternalSecret for each environment

Use the following script to create the SecretStore and ExternalSecret dynamically for each namespace (`dev`, `qa`, `prod`).

```
for ENV in dev qa prod; do
  echo "Deploying SecretStore and ExternalSecret in $ENV..."

  helm upgrade --install vault-$ENV \
    helm/external-secrets/secretStore/ \
    --namespace $ENV \
    --set vault.vaultPath=secret-$ENV \
    --set vault.server=http://vault.vault.svc:8200 \
    --set vault.auth.kubernetes.mountPath=kubernetes \
    --set vault.auth.kubernetes.role=$ENV \
    --set vault.auth.kubernetes.serviceAccountName=vault-sa

  helm upgrade --install sentinel-1-secret-$ENV \
    helm/external-secrets/externalSecret/ \
    --namespace $ENV \
```

```

--set name=keycloak-client-secret \
--set secretStore.name=vault-$ENV-store \
--set secretStore.kind=SecretStore \
--set target.creationPolicy=Owner \
--set data[0].secretKey=client_id \
--set data[0].remoteRef.key=iam/idp_clients/sentinel-1 \
--set data[0].remoteRef.property=client_id \
--set data[1].secretKey=client_secret \
--set data[1].remoteRef.key=iam/idp_clients/sentinel-1 \
--set data[1].remoteRef.property=client_secret
done

```

This loop will:

- Create a **SecretStore** configured for the correct Vault KV engine per environment.
- Create an **ExternalSecret** to sync the **client_id** and **client_secret** for the Sentinel-1 API.

5.3.2. Step 2: Deploy the Sentinel-1 API Across Environments

Once the secrets are synchronized into each namespace, we proceed to deploy the **Sentinel-1** API into the **dev**, **qa**, and **prod** environments.

To automate the deployment process and avoid repetitive manual steps, use the following script:

```

# Loop through each environment and deploy the Sentinel-1 API and its corresponding
Ingress
for ENV in dev qa prod; do
  echo -e "\n Deploying Sentinel-1 API in $ENV... \n"

  # Deploy the API using Helm
  helm upgrade --install sentinel-1 helm/sentinel-1/ \
    --namespace $ENV \
    --set api.name=sentinel-1-$ENV \
    --set api.image.name=wmoinar/sentinel-1-$ENV \
    --set api.image.tag=1.0.0 \
    --set api.redirect_url=http://sentinel-1-$ENV.test/callback \
    --set keycloak.issuer_url=http://keycloak.test/realms/$ENV \
    --set keycloak.internal_url=http://keycloak.iam.svc.cluster.local/realms/$ENV

  # Deploy the Ingress for external access
  helm upgrade --install sentinel-1-$ENV-ingress helm/traefik/ingress/ \
    --namespace $ENV \
    --set annotations.ingressClass=traefik \
    --set spec.ingressClassName=traefik \
    --set spec.rules.host=sentinel-1-$ENV.test \
    --set spec.rules.paths[0].path="/" \
    --set spec.rules.paths[0].pathType=Prefix \
    --set spec.rules.paths[0].backend.service.name=sentinel-1-$ENV \
    --set spec.rules.paths[0].backend.service.port=80

```

```
--set tls.enabled=false  
done
```

This loop automates:

- Deployment of the Sentinel-1 API containers for each environment.
- Creation of the corresponding Ingress resources to expose the APIs externally.

5.3.3. Step 3: Configure Local Access

To access the APIs using their respective domain names, add the following entries to your `/etc/hosts` file:

```
echo "127.0.0.1 sentinel-1-dev.test" | sudo tee -a /etc/hosts  
echo "127.0.0.1 sentinel-1-qa.test" | sudo tee -a /etc/hosts  
echo "127.0.0.1 sentinel-1-prod.test" | sudo tee -a /etc/hosts
```

This ensures that your local machine resolves the custom hostnames to your cluster's Ingress controller.

5.4. Verifying the End-to-End Platform Setup

This section provides a structured validation process to ensure that all deployed components—Vault, External Secrets Operator (ESO), Keycloak, and the Sentinel-1 application—are fully operational and integrated across all target environments (`dev`, `qa`, and `prod`).

5.4.1. Infrastructure and Synchronization Validation

First, validate the state of Kubernetes workloads and resources.

Check that the Sentinel-1 API pods are running:

```
oc get pods -A | grep sentinel-1
```

Expected output:

dev	sentinel-1-dev-7d8df797c8-66htp	1/1	Running	0	11h
prod	sentinel-1-prod-74d897864-hfzqz	1/1	Running	0	11h
qa	sentinel-1-qa-59d6d55795-z8pps	1/1	Running	0	11h

Validate that the `SecretStore` resources have been successfully created and are in a `Ready` state:

```
oc get SecretStore -A
```

Expected output:

NAMESPACE	NAME	AGE	STATUS	CAPABILITIES	READY
dev	vault-dev-store	44h	Valid	ReadWrite	True
dev	vault-store	43h	Valid	ReadWrite	True
prod	vault-prod-store	43h	Valid	ReadWrite	True
qa	vault-qa-store	43h	Valid	ReadWrite	True

Verify that all **ExternalSecret** resources are actively synchronized with Vault:

```
oc get ExternalSecret -A
```

Expected output:

NAMESPACE	NAME	STORETYPE	STORE	REFRESH INTERVAL
STATUS	READY			
dev	dev-credential	SecretStore	vault-store	1h
SecretSynced	True			
dev	sentinel-1-secret-dev	SecretStore	vault-dev-store	1h
SecretSynced	True			
prod	sentinel-1-secret-prod	SecretStore	vault-prod-store	2s
SecretSynced	True			
qa	sentinel-1-secret-qa	SecretStore	vault-qa-store	1h
SecretSynced	True			

Finally, confirm that Kubernetes secrets have been materialized correctly:

```
oc get secret -A | grep synced-sentinel-1-secret
```

Expected output:

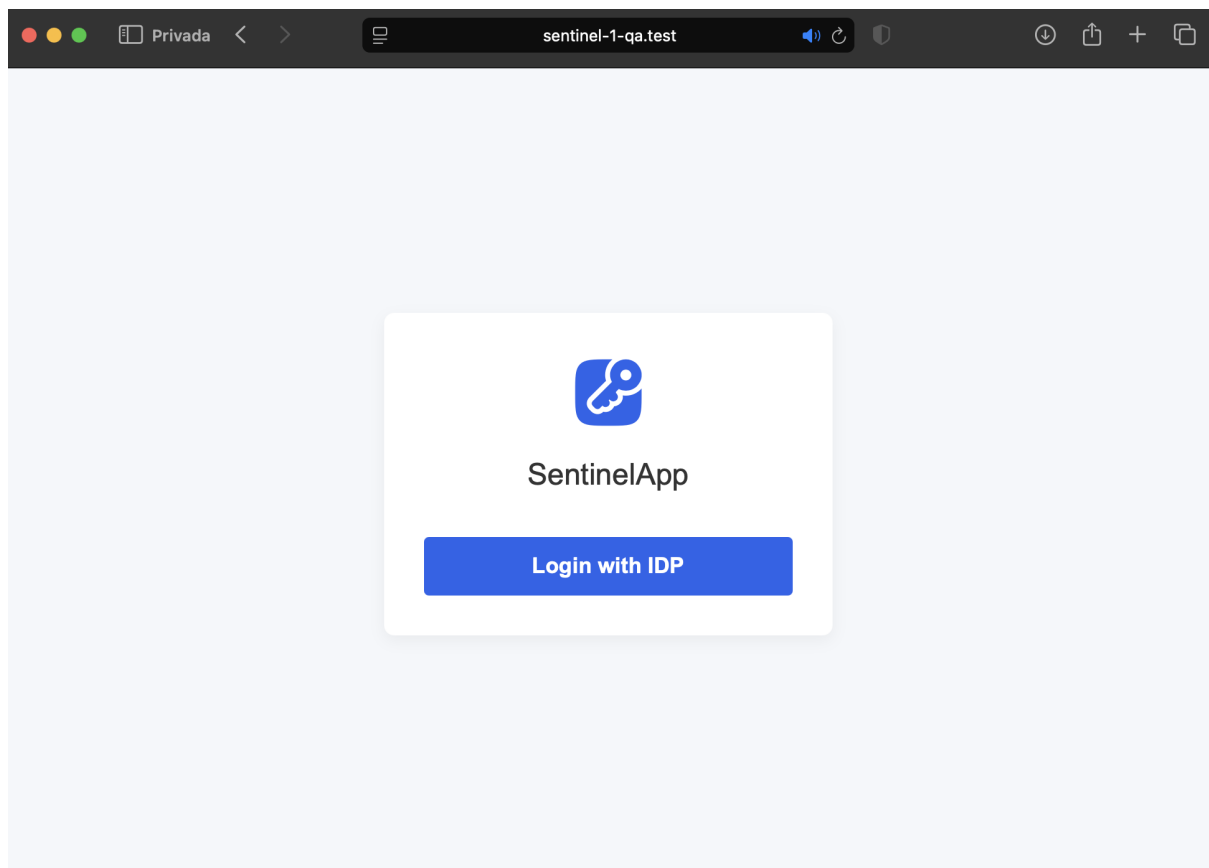
dev	synced-sentinel-1-secret-dev	Opaque	2	12h
prod	synced-sentinel-1-secret-prod	Opaque	2	12h
qa	synced-sentinel-1-secret-qa	Opaque	2	12h

5.4.2. Application Functionality Validation

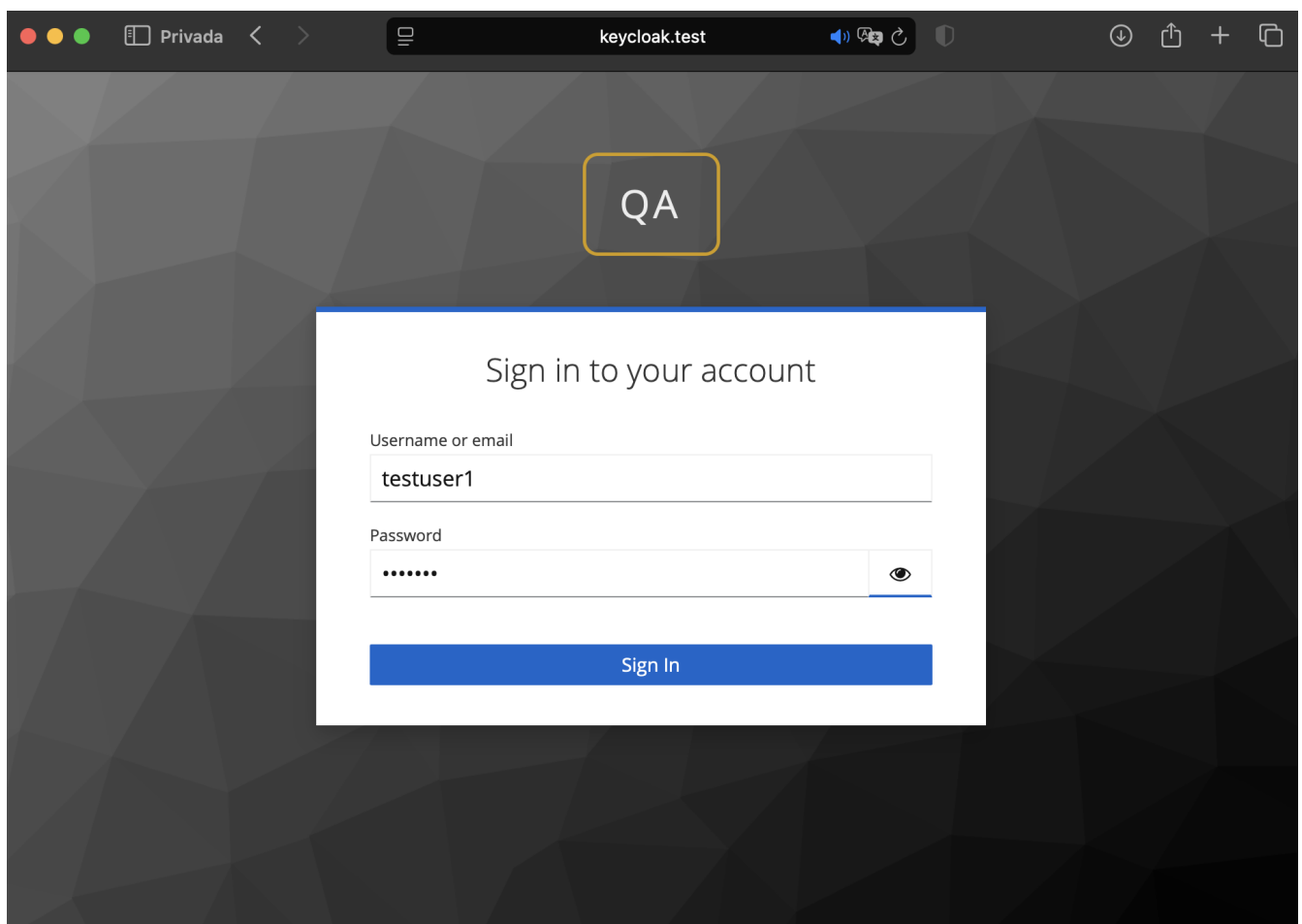
Next, validate end-to-end application login and token acquisition.

1. Open a browser and navigate to the deployed application for the **qa** environment:
<http://sentinel-1-qa.test>

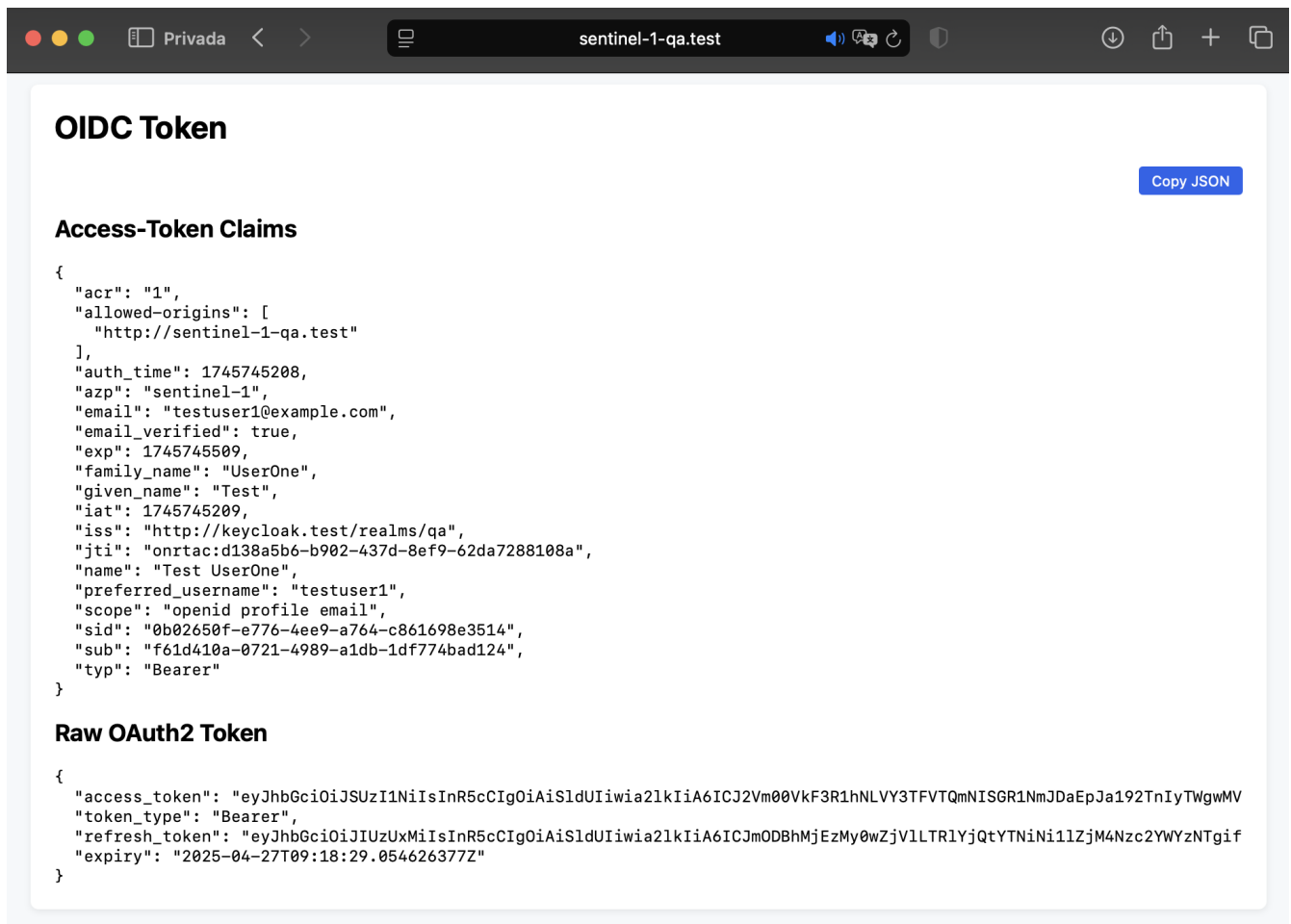
You should see the **Login with Keycloak** button.



1. Click the login button to initiate the OIDC authentication flow via Keycloak. You should be redirected to the Keycloak login page associated with the **qa** realm.



1. Authenticate with valid user credentials (**testuser1** or equivalent).
2. Finally, the Sentinel-1 API will display the retrieved ID token, including decoded claims from the Access Token:



OIDC Token

[Copy JSON](#)

Access-Token Claims

```
{
  "acr": "1",
  "allowed-origins": [
    "http://sentinel-1-qa.test"
  ],
  "auth_time": 1745745208,
  "azp": "sentinel-1",
  "email": "testuser1@example.com",
  "email_verified": true,
  "exp": 1745745509,
  "family_name": "UserOne",
  "given_name": "Test",
  "iat": 1745745209,
  "iss": "http://keycloak.test/realms/qa",
  "jti": "onrtac:d138a5b6-b902-437d-8ef9-62da7288108a",
  "name": "Test UserOne",
  "preferred_username": "testuser1",
  "scope": "openid profile email",
  "sid": "0b02650f-e776-4ee9-a764-c861698e3514",
  "sub": "f61d410a-0721-4989-a1db-1df774bad124",
  "typ": "Bearer"
}
```

Raw OAuth2 Token

```
{
  "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCIgOiAiSldUIiwia2lkIiA6ICJ2Vm00VkF3R1hNLVY3TFVTQmNISGR1NmJDaEpJa192TnIyTWgwMV",
  "token_type": "Bearer",
  "refresh_token": "eyJhbGciOiJIUzUxMiIsInR5cCIgOiAiSldUIiwia2lkIiA6ICJmODBhMjEzMy0wZjV1LTRYjQYTNiNi1lZjM4Nzc2YWYzNTgif",
  "expiry": "2025-04-27T09:18:29.054626377Z"
}
```

5.4.3. Summary

Successful validation across these steps confirms that:

- Vault, Kubernetes, and ESO are correctly integrated and secrets are dynamically provisioned.
- Keycloak is acting as the identity provider with environment-specific realms.
- The Sentinel-1 API is capable of real-time secret consumption and OIDC-based user authentication.



If any validation step fails, investigate the corresponding subsystem (Vault policy, ESO configuration, Keycloak client mapping, or Kubernetes resource deployment) before proceeding.

Chapter 6. Best Practices and Security Considerations

- Never hardcode Vault tokens or root credentials in source files.
- Use short-lived tokens and role-based access control (RBAC) in Vault.
- Ensure TLS is configured in production scenarios.
- Use namespaces to segment teams or environments cleanly.
- Set resource limits and configure alerts for secret rotation failures.

Chapter 7. References

- <https://developer.hashicorp.com/vault/docs>
- <https://external-secrets.io>
- <https://k3d.io>
- <https://helm.sh>
- <https://www.keycloak.org>

Chapter 8. List of figures

list-of::image[]

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
1BmE4kWBq78iYhF1dvKuhfTAU6auU8tT94WrHftjDbrCEXSU1oBoqyl2QvZ6jIW3" crossorigin="anonymous">
```